

Productor-Consumidor sobre Buffer Acotado: Evidencia Cuantificada de una Condición de Carrera y Medición del Costo de su Corrección

Isaac Felipe Morún Moreira

Universidad Invenio — Licenciatura en Tecnologías de Información y Comunicación Empresarial

TIIT2007 – Sistemas Operativos | Agosto 2026

1. Resumen

Este documento presenta el diseño, la implementación y la evaluación experimental de dos versiones de un simulador Productor-Consumidor sobre un buffer acotado, desarrolladas en C++17 con hilos reales. Ambas comparten el mismo diseño lógico, los mismos parámetros y la misma instrumentación; la única diferencia entre ellas es la presencia o ausencia de sincronización. La versión A accede al buffer y a sus variables de estado sin ninguna protección; la versión B emplea exclusión mutua con un mutex y coordina la espera por buffer lleno o vacío con dos variables de condición, sin espera activa. La verificación de invariantes al final de cada corrida compara los ítems producidos y consumidos, la suma de los valores transportados, la ocupación máxima observada frente a la capacidad, y reconcilia los identificadores individuales para cuantificar pérdidas, duplicaciones y corrupción de campos. Sobre cuarenta corridas distribuidas en cuatro configuraciones, con el total de ítems fijo como variable controlada, la versión A violó el invariante en las veinte corridas y la versión B lo satisfizo en las veinte. El sobrecosto de la sincronización, calculado como la variación porcentual del tiempo medio, osciló entre 2 466 % y 13 144 % según la configuración, con una relación inversa clara respecto del tamaño del buffer. La existencia de las condiciones de carrera fue confirmada de forma independiente con ThreadSanitizer, que reportó ocho carreras en la versión A y ninguna en la versión B, y se documenta un intercalado concreto extraído del registro de eventos con su cadena causal completa.

Palabras clave—condición de carrera, sección crítica, exclusión mutua, mutex, variable de condición, invariante, buffer acotado, C++17.

2. Introducción

2.1 Contexto del problema

Cuando dos o más hilos acceden a la misma región de memoria y al menos uno de ellos escribe, sin un mecanismo de ordenamiento el resultado del programa deja de estar definido. Las manifestaciones habituales son actualizaciones perdidas, lecturas de estados intermedios y violación de invariantes estructurales. La dificultad práctica de este fenómeno es que no se manifiesta de forma determinista: el mismo binario, con la misma entrada, puede producir un resultado correcto en una ejecución e incorrecto en la siguiente.

El problema Productor-Consumidor sobre un buffer de capacidad finita es el caso canónico para estudiarlo. Un conjunto de hilos genera elementos y los deposita en una estructura

compartida mientras otro conjunto los retira. La coordinación necesaria no es trivial: los productores deben esperar cuando el buffer está lleno, los consumidores cuando está vacío, y ambos grupos manipulan los mismos índices y el mismo contador de ocupación.

El objetivo de este trabajo no es únicamente implementar una solución correcta, sino observar empíricamente el fallo, cuantificarlo con invariantes verificables y medir lo que cuesta corregirlo bajo un protocolo experimental controlado.

2.2 Objetivo general

Implementar dos versiones de un simulador Productor-Consumidor que difieran exclusivamente en la presencia de sincronización, verificar automáticamente el cumplimiento de invariantes en cada corrida, y comparar ambas versiones bajo un protocolo experimental que permita atribuir las diferencias observadas al mecanismo de sincronización y no a otras variables.

2.3 Objetivos específicos

- Implementar un buffer circular de tamaño finito configurable, con la sección crítica delimitada explícitamente en comentarios en ambas versiones.
- Construir una versión sin sincronización cuya vulnerabilidad sea genuina, sin fabricar el error ni introducir retardos que lo fuercen.
- Diseñar un esquema de verificación de invariantes que incluya una suma de control sobre los valores transportados, y que no introduzca por sí mismo condiciones de carrera en la medición.
- Implementar exclusión mutua correcta con mutex y coordinación con variables de condición, sin espera activa ni posibilidad de interbloqueo.
- Garantizar terminación limpia y determinista sin recurrir a temporizadores ni a la terminación forzada de procesos.
- Registrar un log de eventos que permita reconstruir qué hilo operó sobre qué posición y en qué orden aproximado.
- Ejecutar el protocolo experimental con un mínimo de cinco repeticiones por versión en cada una de cuatro configuraciones, reportando media, desviación estándar, mínimo y máximo.
- Calcular e interpretar el sobrecosto porcentual de la sincronización, reconociendo la limitación

metodológica que supone comparar contra un programa incorrecto.

- Obtener evidencia independiente mediante un detector dinámico de carreras y contrastar su salida entre ambas versiones.

2.4 Alcance y limitaciones

El trabajo modela exclusivamente el patrón Productor-Consumidor sobre un buffer circular en memoria compartida, con hilos del mismo proceso. Los ítems transportados son estructuras sintéticas diseñadas para permitir la verificación de invariantes; no representan una carga aplicativa real y no se realiza cómputo ni entrada/salida dentro de la sección crítica.

Además de las dos versiones exigidas se incluyen, en un directorio separado y claramente identificado como material adicional, dos mecanismos alternativos que también son correctos: uno con mutex y espera activa, y otro con tres semáforos. El primero existe para sostener con datos propios la exigencia de que toda espera activa se documente con su costo medido; el segundo, para contrastar experimentalmente mutex y semáforo. Se incluye asimismo una demostración independiente de interbloqueo que reproduce las cuatro condiciones de Coffman y dos técnicas de prevención; esa demostración se bloquea deliberadamente en su modo por defecto y no guarda relación con la versión B, que no puede interbloquearse.

3. Marco Teórico

3.1 Condición de carrera y sección crítica

Se denomina sección crítica al fragmento de código que accede a un recurso compartido y que debe ejecutarse sin intercalación con otras secciones críticas sobre el mismo recurso [1]. Una condición de carrera ocurre cuando el resultado observable del programa depende del orden relativo en que los hilos ejecutan esas secciones.

El caso elemental es el incremento de un contador compartido: la expresión que aparenta ser una operación única se traduce en tres pasos —leer, modificar, escribir— y dos hilos que los ejecuten de forma intercalada pueden leer el mismo valor inicial, calcular el mismo resultado y escribirlo, perdiendo uno de los dos incrementos sin dejar rastro.

Conviene distinguir dos nociones que suelen confundirse. Una carrera de datos, en la terminología del estándar del lenguaje, es la situación en que dos hilos acceden a la misma posición de memoria sin orden establecido entre ellos y al menos uno escribe; en C++ constituye comportamiento indefinido. Una condición de carrera, en la terminología de sistemas operativos, es la propiedad de que el resultado dependa del orden de ejecución. Toda carrera de datos puede provocar una condición de carrera, pero existen condiciones de carrera sin carrera de datos, por ejemplo cuando dos operaciones individualmente atómicas se ejecutan en un orden imprevisto.

La consecuencia práctica es relevante para este trabajo: dado que la versión A incurre formalmente en comportamiento indefinido, lo que se observa no es «el» comportamiento del programa sino su manifestación concreta bajo un compilador y una arquitectura determinados.

3.2 Exclusión mutua

La exclusión mutua exige que a lo sumo un hilo se encuentre dentro de la sección crítica en cada instante. Una solución correcta debe satisfacer además progreso —si nadie está dentro, algún candidato debe poder entrar— y espera acotada —ningún hilo debe esperar indefinidamente mientras otros entran repetidamente [1]. El mutex implementa directamente este requisito y se caracteriza por tener propietario: el hilo que lo adquiere es el único autorizado a liberarlo, lo que permite razonar localmente sobre la corrección del código protegido.

3.3 Espera bloqueante frente a espera activa

Un mutex resuelve la exclusión mutua pero no la espera por una condición. Si un productor encuentra el buffer lleno debe aguardar a que un consumidor libere espacio. La forma más simple de esperar es consultar la condición en un bucle, pero ese enfoque consume procesador mientras no hay trabajo que realizar.

La variable de condición permite que el hilo se bloquee liberando el mutex de forma atómica y sea despertado cuando otro hilo notifique el cambio. Su uso correcto exige reevaluar la condición al despertar, porque una variable de condición puede producir despertares espurios: retornos de la espera sin que se haya emitido notificación alguna. La forma con predicado gestiona ese caso automáticamente.

3.4 Terminación de los consumidores

Un aspecto del problema que suele pasarse por alto es la terminación. Los productores conocen de antemano cuántos ítems deben generar, pero los consumidores no tienen forma intrínseca de saber cuándo dejar de esperar. Las soluciones habituales son una bandera de fin de producción, una píldora venenosa —un elemento centinela que indica el cierre— o un contador de ítems restantes [2]. Recurrir a un temporizador es inaceptable porque introduce una dependencia arbitraria del tiempo de ejecución y puede tanto cortar antes de tiempo como no cortar nunca.

3.5 Relación con la observación práctica en xv6

El kernel didáctico xv6 protege sus estructuras compartidas mediante spinlocks, implementados en spinlock.c con las funciones acquire() y release(). Un spinlock es exactamente una espera activa: el procesador que no logra adquirir el cerrojo permanece girando hasta conseguirlo en lugar de ceder el procesador. La justificación habitual es que las secciones críticas del kernel son muy breves y el costo de un cambio de contexto superaría al de girar unas pocas iteraciones. El material adicional de este proyecto permite contrastar esa afirmación con mediciones propias, dado que incluye una variante con mutex y espera activa cuya única diferencia respecto de la versión B es precisamente la forma de esperar.

4. Diseño de la Solución

4.1 Arquitectura general

El proyecto se organiza de modo que las dos versiones compilen por separado pero compartan todo lo que no sea el mecanismo

de sincronización. Un directorio común contiene la definición del ítem, la instrumentación con su reconciliación y el volcado del log, y el orquestador que crea los hilos, mide el tiempo y emite la salida. Cada versión aporta únicamente su clase de buffer y un archivo principal de una docena de líneas que la instancia.

El orquestador es una función plantilla y no una jerarquía con métodos virtuales. La razón es de medición: una llamada virtual en el camino de ejecución más frecuente introduciría una indirección que se sumaría al tiempo medido y contaminaría justamente la magnitud que el proyecto se propone cuantificar.

4.2 El ítem y los invariantes

Cada ítem transporta cinco campos con funciones distintas en la verificación. Un valor entero, cuya suma se compara entre producción y consumo, constituye el invariante fuerte que el enunciado solicita. Un identificador único global permite además determinar qué ítems concretos se perdieron y cuáles se duplicaron. Una suma de verificación derivada de los demás campos detecta escrituras entrelazadas campo por campo, es decir, ítems cuyos campos provienen de dos escrituras distintas.

La elección de varios campos en lugar de un único entero responde a una propiedad del hardware: en x86-64 una escritura alineada de ocho bytes es atómica a nivel de instrucción, de modo que con un solo entero nunca podría observarse un valor parcialmente escrito y la corrupción se limitaría a pérdidas y sobrescrituras.

Debe señalarse una limitación del invariante basado en sumas, que motiva mantener también la reconciliación por identificador: una pérdida y una duplicación de valores equivalentes se cancelan mutuamente y dejarían la suma intacta. La suma es condición necesaria de correctitud, no suficiente.

Tabla 1: Métricas obtenidas en la reconciliación posterior a la unión de hilos.

Métrica	Definición operativa
Producidos / Consumidos	Tamaño de los registros privados de cada grupo de hilos
Suma producida / consumida	Suma de los valores entregados y extraídos
Perdidos	Identificadores producidos que no aparecen en ningún consumidor
Duplicados	Identificadores consumidos más de una vez
Fantasma	Identificadores consumidos que ningún productor generó
Corruptos	Ítems cuya suma de verificación no se satisface
Ocupación máxima	Mayor valor del contador observado, y si superó N

La corrida se declara conforme solo si los conteos coinciden, las sumas coinciden, la ocupación nunca superó la capacidad y no hubo inconsistencias de ninguna clase. El programa devuelve además un código de salida distinto según el resultado, lo que permite encadenar corridas en scripts sin analizar la salida de texto.

4.3 Instrumentación sin condiciones de carrera propias

El diseño de la instrumentación está sujeto a una restricción que condiciona la validez de todos los resultados: si los hilos registraran lo producido y lo consumido en una estructura compartida, ese registro sufriría a su vez condiciones de carrera y las cifras obtenidas no probarían nada. La regla adoptada es

que durante la ejecución ningún hilo comparte estructura de medición con otro.

Cada productor acumula en un vector propio los identificadores que entregó y la suma de sus valores; cada consumidor hace lo propio con los ítems que extrajo. La reconciliación y el ordenamiento del log por marca de tiempo se realizan después de unir todos los hilos, en un único hilo de ejecución. El fenómeno medido es no determinista, pero la medición es determinista y auditable. Los vectores se reservan con su capacidad estimada antes de iniciar la medición para que ninguna reasignación de memoria se compute dentro del intervalo cronometrado.

El orden del log resultante es aproximado: dos eventos separados por nanosegundos pueden aparecer invertidos respecto del orden real de ejecución. Es suficiente para reconstruir el intercalado, que es lo que se requiere.

4.4 Terminación determinista sin temporizadores

Los productores generan una cantidad fija conocida de antemano, de modo que terminan por sí mismos; el total esperado se imprime antes de lanzar los hilos. Los consumidores emplean una bandera de fin de producción combinada con la condición de buffer vacío: el hilo principal activa la bandera únicamente después de unir a todos los productores, de manera que la condición de salida no puede cumplirse antes de tiempo. No interviene ningún temporizador.

En la versión B esta construcción es inmediata: la espera sobre la variable de condición incluye la bandera en su predicado y la activación notifica a todos los consumidores dormidos. En la versión A la terminación converge por una razón distinta: una vez activada la bandera ya nadie incrementa el contador de ocupación, de modo que solo puede decrecer, y el bucle de salida termina aun cuando el contador esté corrompido.

4.5 Dos decisiones que deben declararse

Bandera de fin atómica en ambas versiones. La bandera no protege el buffer ni ninguna de sus variables y por tanto no constituye sincronización del recurso compartido; su único propósito es la terminación. Es atómica porque, siendo un valor corriente, su lectura dentro del bucle de espera sería comportamiento indefinido y el compilador podría eliminarla. Está presente por igual en las dos versiones, de modo que no introduce ninguna asimetría entre ellas.

Variables de estado declaradas volatile en la versión A. Conviene ser preciso sobre lo que esto significa. La palabra clave no aporta atomicidad ni ordenamiento de memoria y no es un mecanismo de sincronización: el contador sigue siendo una operación de lectura, modificación y escritura en tres pasos, y la asignación del ítem sigue sin ser atómica. Su único efecto es obligar al compilador a releer la variable en cada iteración. Sin ella, dado que una carrera de datos es comportamiento indefinido, el optimizador está autorizado a suponer que el contador no cambia y a extraer la lectura fuera del bucle de espera, convirtiéndolo en un ciclo infinito; se comprobó que eso ocurre con optimización activada. Es decir, es lo que permite que el programa termine, no lo que permite que falle. La versión B no la necesita porque el mutex ya impone el ordenamiento.

5. Implementación

5.1 Lenguaje, estándar y estructura

El proyecto está escrito en C++17 y utiliza exclusivamente la biblioteca estándar, sin dependencias externas. Compila con un solo comando y cada versión dispone además de su propio archivo de construcción, de modo que puede compilarse de forma independiente.

Tabla 2: Organización del código fuente.

Ubicación	Contenido
common/Item.h	Ítem, valor, suma de verificación, traza
common/Registro.h/.cpp	Instrumentación, reconciliación y log
common/Ejecutor.h	Orquestación, argumentos y salida
version_a/BufferA.h	Buffer sin sincronización
version_b/BufferB.h	Buffer con mutex y variables de condición
herramientas/reporte.cpp	Tablas y análisis desde los CSV
extras/	Material adicional, no exigido

5.2 Versión A: acceso desprotegido

La operación de producción espera a que haya lugar y a continuación escribe. Las cuatro líneas centrales carecen de toda protección y son la única fuente de la corrupción observada.

```
while (cuenta_ >= capacidad_) {  
    std::this_thread::yield();  
}  
// === INICIO SECCIÓN CRÍTICA ===  
const int pos = cabeza_ % capacidad_;  
datos_[pos] = it;  
cabeza_ = (cabeza_ + 1) % capacidad_;  
cuenta_ = cuenta_ + 1;  
// === FIN SECCIÓN CRÍTICA ===
```

El incremento se escribe de forma expandida para que resulten visibles los tres pasos que ejecuta el procesador. El uso de aritmética modular sobre los índices es la propia de un buffer circular y no un mecanismo defensivo, aunque tiene el efecto colateral de que un índice corrompido produzca sobrescritura en lugar de acceso fuera de rango.

Sobre la cesión voluntaria del procesador. La llamada que aparece en el bucle de espera no es un retardo —no fija duración alguna— ni sincronización: no ordena accesos a memoria ni excluye a nadie de la sección crítica. Indica al planificador que el hilo no puede avanzar. Su presencia se justifica con una medición: sin ella, en la configuración de mayor concurrencia y sobre doce corridas, la mediana fue de 10 ms pero dos corridas alcanzaron 21 s y 371 s, porque los hilos que no pueden avanzar consumen el procesador que necesitan los que sí podrían. Con ella, las doce corridas se situaron entre 4.1 ms y 9.1 ms. En esas mismas doce corridas el invariante se violó doce de doce veces, con magnitudes de corrupción equivalentes, de modo que no debilita la evidencia: actúa sobre el bucle de espera, no sobre la sección crítica. La versión B no la incorpora porque no tiene bucles de espera; sus hilos se bloquean y el planificador ya sabe que no son ejecutables.

5.3 Versión B: exclusión mutua y espera bloqueante

La operación de producción toma el cerrojo mediante RAII, espera sobre la variable de condición correspondiente con un predicado, opera y notifica. No existe ningún bucle de consulta.

```
std::unique_lock<std::mutex> cerrojo(mutex_);
```

```
hay_lugar_.wait(cerrojo,  
    [this] { return cuenta_ < capacidad_; });  
// === INICIO SECCIÓN CRÍTICA ===  
datos_[cabeza_ % capacidad_] = it;  
cabeza_ = (cabeza_ + 1) % capacidad_;  
cuenta_ = cuenta_ + 1;  
// === FIN SECCIÓN CRÍTICA ===  
cerrojo.unlock();  
hay_datos_.notify_one();
```

Se emplean dos variables de condición y no una. Los productores esperan la condición «hay lugar» y los consumidores «hay datos», que son condiciones distintas; con una sola variable cada notificación tendría probabilidad apreciable de despertar al tipo de hilo equivocado, que reevaluaría su predicado, lo encontraría falso y volvería a dormirse. La notificación se emite tras liberar el cerrojo, para que el hilo despertado no lo encuentre ocupado y deba bloquearse de nuevo. La única llamada manual a la liberación del cerrojo es precisamente esa, y responde a ese motivo.

La imposibilidad de interbloqueo se sigue de la estructura: existe un único cerrojo, de modo que no puede formarse espera circular, y las esperas sobre las variables de condición liberan el mutex mientras el hilo duerme.

5.4 Compilación y ejecución

```
make                # todo  
make -C version_a   # solo la version A  
make experimentos    # protocolo completo  
make evidencia       # log y carrera explicada  
make tsan            # detector de carreras  
  
./version_b/version_b --capacidad 10 \  
    --productores 4 --consumidores 4 --items 5000
```

Banderas empleadas en todos los binarios: -std=c++17 -Wall -Wextra -pthread -O2. La compilación no produce advertencias.

6. Resultados Experimentales

6.1 Entorno y protocolo

Tabla 3: Entornos de ejecución.

	Entorno 1	Entorno 2
Sistema	Ubuntu 26.04 LTS (VirtualBox)	Windows 11 Pro
Compilador	g++ 15.2.0	g++ 16.2.0 (MinGW-w64)
Núcleos lógicos	6	16
Banderas	-std=c++17 -Wall -Wextra -pthread -O2	idénticas

Se evaluaron cuatro configuraciones, variando el tamaño del buffer y el número de hilos, con cinco repeticiones por versión en cada una: cuarenta corridas por entorno. El total de ítems se mantuvo fijo en 20 000 como variable controlada, de modo que la cantidad de ítems por productor se ajusta al número de productores. Todas las corridas de un entorno se ejecutaron consecutivamente sobre el mismo binario, sin otras cargas pesadas. Salvo indicación contraria, las cifras corresponden al Entorno 1.

6.2 Resultados por corrida

La Tabla 4 recoge las cuarenta corridas del Entorno 1. En todas ellas el total producido fue de 20 000 ítems.

Tabla 4: Corridas con N = 5, 2 productores, 2 consumidores.

#	Ver	Consum.	Suma prod.	Suma cons.	OcMáx	Inv.	ms
1	A	19273	100010000	97629192	6	VIOL	7.125
2	A	22242	100010000	109856519	6	VIOL	10.214
3	A	21388	100010000	106084682	6	VIOL	7.964
4	A	19861	100010000	98307070	6	VIOL	11.939
5	A	20071	100010000	101089378	6	VIOL	5.916
1	B	20000	100010000	100010000	5	OK	1281.860
2	B	20000	100010000	100010000	5	OK	1728.590
3	B	20000	100010000	100010000	5	OK	567.864
4	B	20000	100010000	100010000	5	OK	810.232
5	B	20000	100010000	100010000	5	OK	1327.320

Tabla 5: Corridas con N = 5, 4 productores, 4 consumidores.

#	Ver	Consum.	Suma prod.	Suma cons.	OcMáx	Inv.	ms
1	A	17505	50010000	45194166	8	VIOL	6.178
2	A	18701	50010000	47286775	8	VIOL	9.549
3	A	17668	50010000	45474272	7	VIOL	6.034
4	A	16688	50010000	44046033	8	VIOL	8.997
5	A	16109	50010000	42037754	7	VIOL	8.718
1	B	20000	50010000	50010000	5	OK	379.863
2	B	20000	50010000	50010000	5	OK	906.023
3	B	20000	50010000	50010000	5	OK	845.792
4	B	20000	50010000	50010000	5	OK	139.531
5	B	20000	50010000	50010000	5	OK	731.562

Tabla 6: Corridas con N = 50, 2 productores, 2 consumidores.

#	Ver	Consum.	Suma prod.	Suma cons.	OcMáx	Inv.	ms
1	A	24959	100010000	127337720	52	VIOL	4.187
2	A	23597	100010000	123665725	51	VIOL	2.966
3	A	23216	100010000	107574473	51	VIOL	4.890
4	A	21361	100010000	109177103	51	VIOL	4.769
5	A	22536	100010000	121063973	52	VIOL	8.734
1	B	20000	100010000	100010000	50	OK	141.984
2	B	20000	100010000	100010000	50	OK	103.125
3	B	20000	100010000	100010000	50	OK	157.581
4	B	20000	100010000	100010000	50	OK	149.049
5	B	20000	100010000	100010000	50	OK	177.122

Tabla 7: Corridas con N = 50, 4 productores, 4 consumidores.

#	Ver	Consum.	Suma prod.	Suma cons.	OcMáx	Inv.	ms
1	A	22909	50010000	59112094	52	VIOL	7.329
2	A	28894	50010000	72135963	52	VIOL	11.348
3	A	18202	50010000	51617556	53	VIOL	5.988
4	A	20490	50010000	58338226	53	VIOL	4.691
5	A	22981	50010000	60107807	52	VIOL	5.520
1	B	20000	50010000	50010000	50	OK	128.073
2	B	20000	50010000	50010000	50	OK	174.590
3	B	20000	50010000	50010000	50	OK	142.896
4	B	20000	50010000	50010000	50	OK	189.432
5	B	20000	50010000	50010000	50	OK	259.842

La versión A violó el invariante en las veinte corridas y la versión B lo satisfizo en las veinte, sin excepción en ninguna configuración. Los tres criterios de fallo se manifestaron simultáneamente en la versión A: los conteos difieren, las sumas de control difieren y la ocupación superó la capacidad en todas las configuraciones.

Tabla 8: Corrupción media de la versión A por configuración.

Config.	Perd.	Dupl.	Corr.	OcMáx
N=5, 2P/2C	4 712	5 019	259	6
N=5, 4P/4C	7 147	4 135	346	8
N=50, 2P/2C	6 094	9 108	120	52
N=50, 4P/4C	6 802	9 345	151	53

La columna de ítems corruptos merece atención: son ítems cuya suma de verificación no se satisface, es decir, cuyos campos

proviene de dos escrituras distintas entrelazadas. No se trata de un error de conteo sino de datos que ningún productor generó.

6.3 Resumen estadístico

Tabla 9: Tiempo de ejecución por versión y configuración. Cinco corridas cada una.

Ver	Config.	Media	Desv.	Mín	Máx	Viol.
A	5,2,2	8.63	2.43	5.92	11.94	5/5
B	5,2,2	1143.17	457.63	567.86	1728.59	0/5
A	5,4,4	7.90	1.66	6.03	9.55	5/5
B	5,4,4	600.55	328.58	139.53	906.02	0/5
A	50,2,2	5.11	2.17	2.97	8.73	5/5
B	50,2,2	145.77	27.23	103.13	177.12	0/5
A	50,4,4	6.98	2.63	4.69	11.35	5/5
B	50,4,4	178.97	51.38	128.07	259.84	0/5

Los tiempos se expresan en milisegundos y la desviación estándar es muestral. La dispersión de la versión B es considerable, en particular con buffer pequeño, donde la desviación alcanza el 40 % de la media: con N = 5 casi toda operación debe bloquearse, y el tiempo pasa a depender de la política de planificación del sistema más que del programa.

6.4 Condición de carrera capturada

El registro de eventos de una corrida reducida —N = 5, dos productores y dos consumidores, 25 ítems por productor— permite exhibir el fenómeno con su cadena causal. El extracto siguiente reproduce las columnas de instante, hilo, operación, identificador, valor, posición y ocupación observada antes y después.

```

ms  hilo tipo      id  val pos  cuenta
0.3494 P0  PROD  1000006  7   1   4->5
0.3495 P1  PROD  2000001  2   1   4->6
0.3496 C0  CONS  1000004  5   4   5->5 <<<
0.3497 C0  CONS  2000000  1   0   5->4
...
0.3504 C0  CONS  1000004  5   4   5->4 <<<

```

Tres hechos independientes son visibles en este fragmento. El primero: los productores P0 y P1 escriben ambos en la posición 1 en instantes consecutivos, de modo que el segundo sobrescribe al primero y ese ítem se pierde. El segundo: la operación marcada a los 0.3496 ms es un consumo tras el cual la ocupación permanece en 5; el decremento se perdió porque otro hilo leyó el mismo valor del contador, operó sobre él y una de las dos escrituras pisó a la otra.

El tercero es consecuencia del segundo. Los decrementos perdidos dejan el contador de ocupación por encima del valor real. Los consumidores creen entonces que hay datos disponibles cuando el buffer ya se vació, avanzan el índice de lectura dando la vuelta al anillo y vuelven a leer una posición que ningún productor ha sobrescrito todavía: el ítem 1000004 se entrega por segunda vez desde la misma posición 4. Ese es el mecanismo por el cual la suma de los valores consumidos llega a superar a la de los producidos, como muestra la Tabla 4. No se inventan ítems: algunos se contabilizan dos veces.

En la corrida completa de la que procede el extracto se registraron 19 ítems consumidos más de una vez y 11 operaciones tras las cuales la ocupación no varió. El log completo y su análisis quedan versionados en el directorio de evidencia.

6.5 Verificación independiente

Ambas versiones se recompilaron con ThreadSanitizer, un detector dinámico que instrumenta los accesos a memoria y reporta las carreras indicando archivo, línea y los dos accesos en conflicto. La versión A produjo ocho reportes distintos; la versión B, ninguno, completando además la totalidad de sus ítems bajo instrumentación. Las líneas señaladas en la versión A corresponden exactamente a las comprobaciones de las condiciones de lleno y vacío y a las escrituras de los índices y del contador, es decir, al patrón de comprobar y actuar descrito en la Sección 3.1.

El detector introduce una penalización de rendimiento de entre cinco y quince veces, por lo cual los binarios instrumentados se emplearon exclusivamente para obtener esta evidencia y ninguna medición de tiempo de este documento proviene de ellos.

6.6 Sobrecosto de la sincronización

Tabla 10: Sobrecosto porcentual sobre el tiempo medio, ambos entornos.

Config.	Entorno 1	Entorno 2
N=5, 2P/2C	13 144 %	2 750 %
N=5, 4P/4C	7 507 %	3 204 %
N=50, 2P/2C	2 753 %	335 %
N=50, 4P/4C	2 466 %	641 %

El sobrecosto es de uno a dos órdenes de magnitud y presenta una relación inversa clara con el tamaño del buffer, que se reproduce en los dos entornos: multiplicar por diez la capacidad reduce el sobrecosto entre cuatro y cinco veces. La interpretación se desarrolla en la Sección 8.1.

Advertencia metodológica. El tiempo de la versión A no constituye una referencia válida en sentido estricto, porque corresponde a un programa incorrecto. La versión A no realiza el mismo trabajo que la versión B: pierde alrededor de un tercio de los ítems y entrega otros por duplicado, de modo que parte de su rapidez procede precisamente de no coordinar. El cociente debe leerse como el precio medido de pasar de un resultado incorrecto a uno correcto, y no como una comparación entre dos soluciones equivalentes. Esta limitación es intrínseca al diseño experimental y no puede eliminarse normalizando por ítem entregado, porque los ítems que la versión A entrega mal también consumieron tiempo.

6.7 Verificación en una segunda plataforma

El código compiló sin modificaciones y sin advertencias en ambos entornos, que difieren en el sistema operativo y en la versión mayor del compilador. El protocolo completo se repitió en el Entorno 2 con resultados cualitativamente idénticos: la versión A violó el invariante en las veinte corridas y la versión B lo satisfizo en las veinte.

Las magnitudes sí difieren. El sobrecosto resultó entre dos y ocho veces menor en el Entorno 2, y la dispersión de los tiempos fue notablemente más baja: la desviación estándar de la versión B no superó el 8 % de la media, frente al 40 % observado en el entorno virtualizado. Ambas diferencias son coherentes con la disponibilidad de procesadores, dieciséis frente a seis, y con la ausencia de una capa de virtualización.

Durante esta verificación se resolvió además una duda de portabilidad: la bandera de compilación para hilos, necesaria en

el primer entorno, resultó inocua en el segundo. Se comprobó comparando los binarios generados con y sin ella, que presentan idéntico tamaño e idéntica lista de bibliotecas dinámicas, incluida la de hilos POSIX, que se enlaza en ambos casos. Se conserva por portabilidad.

7. Evaluación ISO/IEC 25010

Autoevaluación conforme a la rúbrica institucional, en escala de Excelente (4) a Insuficiente (1), justificada con evidencia propia del proyecto.

Tabla 11: Autoevaluación ISO/IEC 25010.

Característica	C.	Justificación
Rendimiento	4	Se mide el tiempo con reloj monótono y se reporta en milisegundos con media, desviación estándar, mínimo y máximo sobre cinco repeticiones en cada una de cuatro configuraciones y dos entornos (Tablas 9 y 10). El sobrecosto se calcula con la fórmula del protocolo y se interpreta, no solo se enuncia. La instrumentación reserva memoria antes de cronometrar para no contabilizar reasignaciones.
Fiabilidad	4	La versión B satisfizo el invariante en 40 de 40 corridas entre ambos entornos, resultado confirmado de forma independiente con ThreadSanitizer. El programa valida los argumentos, rechaza configuraciones que incumplen el mínimo de dos productores y dos consumidores, y devuelve un código de salida distinto según el resultado de la verificación. La terminación no depende de temporizadores.
Usabilidad	3	La interfaz ofrece ayuda en línea, mensajes de error específicos por argumento, imprime el total esperado antes de arrancar y resalta en color los invariantes violados. No alcanza el máximo porque la interpretación de las métricas de salida —en particular la distinción entre ítems fantasma y corruptos— requiere consultar la documentación: la salida no es autoexplicativa para quien desconozca el diseño del verificador.
Compatibilidad	4	Compila y ejecuta sin modificaciones ni advertencias en dos sistemas operativos y dos versiones mayores del compilador, con el protocolo experimental completo repetido en ambos (Sección 6.7). Se verificó experimentalmente el comportamiento de la bandera de hilos en cada plataforma comparando binarios y dependencias. El generador de reportes ofrece un modo cuya salida se comprobó libre de caracteres fuera del rango ASCII.
Mantenibilidad	4	El código común a ambas versiones está factorizado en un directorio compartido, de modo que la única diferencia entre ellas es la clase de buffer; agregar un mecanismo nuevo no requiere tocar la orquestación ni la instrumentación, como demuestran los dos mecanismos adicionales incluidos. Las secciones críticas están delimitadas con comentarios explícitos. Los archivos de construcción declaran los encabezados como dependencia, tras detectarse que su omisión hacía medir binarios desactualizados.

La única calificación inferior al máximo corresponde a una limitación real y verificable. Se ha evitado afirmar cualquier propiedad que no cuente con un comando capaz de reproducirla.

8. Discusión de Resultados

8.1 Interpretación

En el eje de la corrección el resultado es categórico y no admite matices: la versión sin sincronización falla en la totalidad de las corridas, en las cuatro configuraciones y en los dos entornos, y la versión sincronizada no falla en ninguna. No existe compensación posible en la que perder un tercio de los datos resulte aceptable.

En el eje del rendimiento, el hallazgo más informativo no es la magnitud del sobrecosto sino su dependencia del tamaño del buffer. Con N = 5 el buffer se encuentra casi permanentemente lleno o vacío, de modo que prácticamente toda operación debe bloquearse: cada una paga una llamada al sistema y un cambio de contexto, del orden de microsegundos, frente a un trabajo útil del orden de nanosegundos. Con N = 50 el buffer absorbe los

desajustes momentáneos entre producción y consumo, las esperas se vuelven infrecuentes y el sobre costo cae entre cuatro y cinco veces. Es decir, buena parte de lo que se atribuye al costo del mutex es en realidad el costo de bloquearse, y ese costo se controla dimensionando el buffer.

La dispersión aporta una segunda lectura. La desviación estándar de la versión B alcanza el 40 % de su media en el entorno virtualizado y no supera el 8 % en el nativo, mientras que la versión A mantiene dispersiones bajas en ambos. Un programa que se bloquea queda a merced de la política de planificación; uno que nunca se bloquea, no. Esto no favorece a la versión A: su determinismo temporal se obtiene a costa de la corrección.

8.2 Relación con la teoría

La ubicación de la comprobación explica toda la corrupción. La diferencia entre ambas versiones no reside en la cantidad de código ni en su complejidad, sino en si la comprobación de la condición y la acción correspondiente ocurren dentro del mismo bloque. El detector dinámico confirma esta lectura de forma independiente al señalar como par en conflicto precisamente las líneas de comprobación y de actualización del contador. Es la manifestación empírica de que la sección crítica no es un conjunto de instrucciones peligrosas sino un intervalo que debe permanecer indivisible.

La corrupción observada corresponde a los modos previstos. Los tres mecanismos anticipados en el diseño se manifestaron en las proporciones esperables: pérdidas por sobrescritura, duplicaciones por relectura de posiciones no repuestas y corrupción de campos por escrituras entrelazadas. La cuarta manifestación, la violación de la capacidad, se observó en todas las configuraciones, con ocupaciones máximas de 6 y 8 sobre un límite de 5 y de 52 y 53 sobre un límite de 50. Es la evidencia más directa del hueco entre comprobar y actuar: el hilo verifica que hay lugar, otro llena el buffer, y el primero escribe igualmente porque ya pasó la comprobación.

El comportamiento indefinido no es una abstracción del estándar. La versión A incurre formalmente en él, y ello tuvo una consecuencia concreta durante el desarrollo: sin declarar volátiles las variables de estado, el optimizador convertía el bucle de espera en un ciclo infinito. Lo observado no es, por tanto, el comportamiento del programa sino su manifestación bajo un compilador y una arquitectura determinados. Que los resultados cualitativos se reproduzcan en dos compiladores distintos refuerza la conclusión, pero no la convierte en una propiedad del lenguaje.

8.3 Limitaciones

El tiempo de la versión A no es una referencia válida. Es la limitación principal y es intrínseca al diseño experimental: se compara contra un programa incorrecto que no realiza el mismo trabajo. Tampoco puede corregirse normalizando por ítem correctamente entregado, porque los ítems mal entregados también consumieron tiempo de ejecución.

La suma de control no es un invariante suficiente. Una pérdida y una duplicación de valores equivalentes se cancelarían y dejarían la suma intacta. Por eso se mantiene además la reconciliación por identificador único, que sí distingue ambos

casos. En las corridas realizadas no se observó ninguna cancelación de este tipo, pero su ausencia es un hecho empírico y no una garantía.

Los tiempos absolutos no son reproducibles entre sesiones en el entorno virtualizado. Las comparaciones se realizan dentro de una misma tanda y se reportan con su dispersión; ninguna cifra absoluta debe interpretarse como característica del sistema.

La carga de trabajo es sintética y homogénea. Todos los productores generan a la máxima velocidad posible y sin cómputo asociado. Una carga con tiempos variables o con trabajo dentro de la sección crítica alteraría el perfil de contención y probablemente reduciría el sobre costo relativo, al aumentar el trabajo útil por cada bloque.

No se midió la equidad entre hilos. El diseño no registra cuántos ítems procesó cada hilo individualmente, de modo que no puede evaluarse la propiedad de espera acotada enunciada en la Sección 3.2 ni descartarse que alguna versión favorezca sistemáticamente a determinados hilos.

La verificación con el detector dinámico se realizó con carga reducida. Se emplearon 500 ítems por productor en lugar de los 5 000 o 10 000 del protocolo, debido a la penalización de rendimiento del instrumento. La ausencia de hallazgos en la versión B es evidencia sólida pero no constituye una prueba de corrección para toda ejecución posible.

9. Conclusiones

El proyecto produjo dos implementaciones del problema Productor-Consumidor que comparten diseño, parámetros e instrumentación y difieren exclusivamente en la presencia de sincronización, junto con un esquema de verificación de invariantes cuya validez descansa en que ningún hilo comparte estructuras de medición durante la ejecución. Esa decisión es la que permite sostener que el fenómeno medido es no determinista mientras la medición es determinista y auditable.

El resultado central es inequívoco: la versión sin sincronización violó el invariante en las cuarenta corridas realizadas entre ambos entornos, con las cuatro manifestaciones previstas — pérdidas, duplicaciones, corrupción de campos y violación de la capacidad— presentes simultáneamente, mientras que la versión sincronizada lo satisfizo en las cuarenta sin excepción.

Tres aprendizajes se derivan del desarrollo. El primero es que la corrección en concurrencia no depende de la cantidad de código protector sino de la ubicación exacta de la frontera de la sección crítica: la diferencia entre una implementación que corrompe un tercio de sus datos y otra que no falla nunca es que la comprobación de la condición ocurra dentro o fuera del mismo bloque.

El segundo es que el sobre costo de la sincronización no es una constante del mecanismo sino una función de la contención. Al multiplicar por diez la capacidad del buffer, manteniendo todo lo demás igual, el sobre costo cayó entre cuatro y cinco veces en ambos entornos. Buena parte de lo que suele atribuirse al costo del mutex es en realidad el costo de bloquearse, y ese costo se controla dimensionando la estructura compartida.

El tercero es de método. La exigencia de que el programa terminara sin temporizadores obligó a razonar sobre la terminación como un problema de diseño y no como un detalle de implementación, y condujo a una construcción —bandera de fin activada después de unir a los productores— que resulta correcta incluso cuando el contador de ocupación está corrompido. Reconocer explícitamente que el tiempo de un programa incorrecto no es una referencia válida, en lugar de presentar el cociente como una comparación limpia, forma parte del mismo criterio.

Como extensiones fuera del alcance se identifican: medir la equidad entre hilos para evaluar empíricamente la espera acotada; incorporar el problema de Lectores-Escritores para contrastar un patrón con concurrencia de lectura; y modelar tiempos de producción y consumo variables para observar cómo se desplaza el punto en que el bloqueo deja de ser el término dominante del costo.

10. Referencias

- [1] A. Silberschatz, P. B. Galvin, y G. Gagne, *Operating System Concepts*, 10.^a ed. Hoboken, NJ, EE. UU.: Wiley, 2018.
- [2] A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.^a ed. Boston, MA, EE. UU.: Pearson, 2015.
- [3] R. H. Arpaci-Dusseau y A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Madison, WI, EE. UU.: Arpaci-Dusseau Books, 2018.
- [4] ISO/IEC, *ISO/IEC 25010:2011 — Systems and software engineering — SQuaRE — System and software quality models*. Ginebra, Suiza: ISO, 2011.